Data Structures (CSN12101)
ASSIGNMENT-10

Name : Aayansh Niranjan Date :14/04/2026
Reg_No.: 20253004

Q1. Write a C program to construct the binary tree if preorder and in-order traversal is given. Print the elements in sorted order. Construct the Binary tree and also find postorder traversal.

A1. CODE:

```
#include<stdio.h>
#include<stdlib.h>

struct Node{int d; struct Node *l,*r;};

int find(int a[],int s,int e,int x){
    for(int i=s;i<=e;i++) if(a[i]==x) return i;
}

struct Node* build(int pre[],int in[],int s,int e,int *pi){
    if(s>e) return NULL;
    struct Node* n=malloc(sizeof(*n));
    n->d=pre[(*pi)++]; n->l=n->r=NULL;
    int i=find(in,s,e,n->d);
    n->l=build(pre,in,s,i-1,pi);
    n->r=build(pre,in,i+1,e,pi);
    return n;
}

void post(struct Node* r){
    if(r){ post(r->l); post(r->r); printf("%d ",r->d); }
}

int main(){
    int in[]={4,2,1,7,5,8,3,6}, pre[]={1,2,4,3,5,7,8,6}, n=8, i=0;
    struct Node* r=build(pre,in,0,n-1,&i);
    post(r);
    printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_10$ ./p1
4 2 7 8 5 6 3 1
```

Q2. Write a C program to construct the binary tree if postorder and in-order traversal is given. Print the elements in sorted order.
A2. CODE:

```
#include<stdio.h>
#include<stdlib.h>

struct Node{int d; struct Node *l,*r;};
int find(int a[],int s,int e,int x){
    for(int i=s;i<=e;i++) if(a[i]==x) return i;
}
struct Node* build(int post[],int in[],int s,int e,int *pi){
    if(s>e) return NULL;
    struct Node* n=malloc(sizeof(*n));
    n->d=post[(*pi)--]; n->l=n->r=NULL;
    int i=find(in,s,e,n->d);
    n->r=build(post,in,i+1,e,pi);
    n->l=build(post,in,s,i-1,pi);
    return n;
}
void inorder(struct Node* r){
    if(r){ inorder(r->l); printf("%d ",r->d); inorder(r->r); }
}
int main(){
    int in[]={4,2,1,7,5,8,3,6}, post[]={4,2,7,8,5,6,3,1}, n=8, i=n-1;
    struct Node* r=build(post,in,0,n-1,&i);
    inorder(r);
    printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_10$ ./p2
4 2 1 7 5 8 3 6
```

Q3. Write a C program to implement heap sort on a given tree:
A3. CODE:

```
#include<stdio.h>

void heapify(int a[],int n,int i){
    int l=2*i+1,r=2*i+2,m=i;
    if(l<n&&a[l]>a[m]) m=l;
    if(r<n&&a[r]>a[m]) m=r;
    if(m!=i){ int t=a[i];a[i]=a[m];a[m]=t; heapify(a,n,m); }
}

void sort(int a[],int n){
    for(int i=n/2-1;i>=0;i--) heapify(a,n,i);
    for(int i=n-1;i>0;i--){
        int t=a[0];a[0]=a[i];a[i]=t;
        heapify(a,i,0);
    }
}

int main(){
    int a[]={5,3,8,4,1,2},n=6;
    sort(a,n);
    for(int i=0;i<n;i++) printf("%d ",a[i]);
    printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_10$ ./p3
1 2 3 4 5 8
```

Q4.WAP to implement Binary Search Tree (BST).

- Insert a node into the BST.
- Delete a node from the BST.
- Display the preorder traversal of the BST.
- Display the inorder traversal of the BST.
- Display the postorder traversal of the BST.

A4. CODE:

```
#include<stdio.h>
#include<stdlib.h>

struct Node{int d; struct Node *l,*r;};
struct Node* ins(struct Node* r,int x){
   if(!r){ r=malloc(sizeof(*r)); r->d=x; r->l=r->r=NULL; return r; }
   if(x<r->d) r->l=ins(r->l,x);
   else r->r=ins(r->r,x);
   return r;
}
struct Node* min(struct Node* r){
   while(r->l) r=r->l; return r;
}
struct Node* del(struct Node* r,int x){
   if(!r) return r;
   if(x<r->d) r->l=del(r->l,x);
   else if(x>r->d) r->r=del(r->r,x);
   else{
      if(!r->l) return r->r;
      if(!r->r) return r->l;
      struct Node* t=min(r->r);
      r->d=t->d;
      r->r=del(r->r,t->d);
   }
   return r;
}
void in(struct Node* r){ if(r){ in(r->l); printf("%d ",r->d); in(r->r);} }
void pre(struct Node* r){ if(r){ printf("%d ",r->d); pre(r->l); pre(r->r);} }
void post(struct Node* r){ if(r){ post(r->l); post(r->r); printf("%d ",r->d);} }
int main(){
   struct Node* r=NULL;
   r=ins(r,50); ins(r,30); ins(r,70); ins(r,20); ins(r,40);
   r=del(r,30);
   in(r); printf("\n"); pre(r); printf("\n"); post(r);
   printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_10$ ./p4
20 40 50 70
50 40 20 70
20 40 70 50
```

Q5. WAP to implement right skewed BST and display the inorder traversal of the tree.
A5. CODE:

```
#include<stdio.h>
#include<stdlib.h>\
struct Node{int d; struct Node* r;};
struct Node* ins(struct Node* root,int x){
   struct Node* n=malloc(sizeof(*n)); n->d=x; n->r=NULL;
   if(!root) return n;
   struct Node* t=root;
   while(t->r) t=t->r;
   t->r=n;
   return root;
}
void in(struct Node* r){
   while(r){ printf("%d ",r->d); r=r->r; }
}
int main(){
   int a[]={1,2,3,4,5}; struct Node* r=NULL;
   for(int i=0;i<5;i++) r=ins(r,a[i]);
   in(r);
   printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_10$ ./p5
1 2 3 4 5
```

Q6. WAP to implement left skewed BST and display the postorder traversal of the tree.
A6. CODE:

```
#include<stdio.h>
#include<stdlib.h>

struct Node{int d; struct Node* l;};

struct Node* ins(struct Node* r,int x){
    struct Node* n=malloc(sizeof(*n));
    n->d=x; n->l=r;
    return n;
}

void post(struct Node* r){
    if(r){ post(r->l); printf("%d ",r->d); }
}

int main(){
    int a[]={1,2,3,4,5}; struct Node* r=NULL;
    for(int i=0;i<5;i++) r=ins(r,a[i]);
    post(r);
    printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_10$ ./p6
1 2 3 4 5
```

Q7. WAP to implement the following:
Count the number of nodes in a binary tree.
Count the number of leaf nodes in a binary tree.
Count the number of non-leaf nodes in a binary tree.
Return the height of the binary tree.
Check whether the tree is a strict binary tree or not.
Check whether the two trees are equal or not.

A7. CODE:

```
#include<stdio.h>
#include<stdlib.h>

struct Node{int d; struct Node *l,*r;};

struct Node* n(int x){
   struct Node* t=malloc(sizeof(*t));
   t->d=x; t->l=t->r=NULL; return t;
}

int cnt(struct Node* r){ return r?1+cnt(r->l)+cnt(r->r):0; }

int leaf(struct Node* r){
   return !r?0:(!r->l&&!r->r)?1:leaf(r->l)+leaf(r->r);
}

int nonleaf(struct Node* r){
   return (!r||(!r->l&&!r->r))?0:1+nonleaf(r->l)+nonleaf(r->r);
}

int h(struct Node* r){
   if(!r) return -1;
   int l=h(r->l), rr=h(r->r);
   return 1+(l>rr?l:rr);
}

int strict(struct Node* r){
   if(!r) return 1;
   if((r->l&&!r->r)||(!r->l&&r->r)) return 0;
   return strict(r->l)&&strict(r->r);
}

int main(){
   struct Node* r=n(1); r->l=n(2); r->r=n(3);
   printf("%d %d %d %d %d",cnt(r),leaf(r),nonleaf(r),h(r),strict(r));
   printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_10$ ./p7
3 2 1 1 1
```